

Sliding Window & Kadane's Algorithm



What We Will Cover

01

Subarrays & Brute Force

Why the naive $O(n^2)/O(n^3)$ approach struggles, and what a subarray really is.

02

Fixed-Size Sliding Window

Maintain a window of exactly K elements in $O(n)$ time.

03

Variable-Size Sliding Window

Grow and shrink the window based on a condition — for longest/shortest subarrays.

04

Kadane's Algorithm

Find the maximum sum subarray in a single $O(n)$ pass, even with negatives.

05

Practice + Homework

Apply everything on classic problems, then 5–8 problems to solve on your own.

What Is a Subarray?

A subarray is a **contiguous** part of an array — no skipping elements, no reordering.

Array A

4	2	-1	7	3	5
0	1	2	3	4	5

2	-1	7
---	----	---

✓ Valid subarray: [2, -1, 7] (elements 1..3, contiguous)

✗ Not a subarray: [4, -1, 5] (skips elements — this is a subsequence)

Why Not Brute Force?

An array of size n has $\frac{n(n+1)}{2}$ subarrays.

Checking every one, and summing each, costs $O(n^2)$ or worse.

$n = 10^5 \rightarrow \sim 5 \times 10^9$
operations

*This times out in almost every judge
(limit $\approx 10^8$ ops/sec).*

Sliding Window & Kadane's turn this
into a single $O(n)$ pass — today's topic.

Fixed-Size Sliding Window — Theory

Idea: keep a window of exactly K elements. Instead of recomputing the sum for each window from scratch, slide it by one step: remove the element leaving, add the element entering.

1

1. Build the first window

Sum the first K elements.

2

2. Slide one step at a time

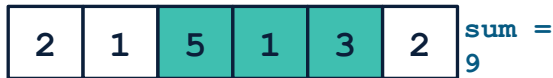
$\text{new_sum} = \text{old_sum} - A[\text{left}] + A[\text{right}]$

3

3. Track the answer

Compare/update as the window moves.

K = 3 Window Sliding



Each slide: $-A[\text{left}] + A[\text{right}]$

Complexity: $O(n)$ time, $O(1)$ extra space — each element enters and leaves the window exactly once.

Worked Example — Max Sum of Size K

Array: [2, 1, 5, 1, 3, 2] $K = 3$ Find the maximum sum of any window of size K .

2	1	5	1	3	2
---	---	---	---	---	---

Window 1: [2,1,5] $\rightarrow$ sum = 8

2	1	5	1	3	2
---	---	---	---	---	---

Window 2: $8 - 2 + 1 = 7$

2	1	5	1	3	2
---	---	---	---	---	---

Window 3: $7 - 1 + 3 = 9$

2	1	5	1	3	2
---	---	---	---	---	---

Window 4: $9 - 5 + 2 = 6$

Maximum sum = 9 (window [5, 1, 3])

Only 4 additions/subtractions total for 4 windows — not $4 \times 3 = 12$.

Fixed-Size Sliding Window — C++ Code

```
int maxSumK(vector<int>& a, int k) {
    int n = a.size();
    int windowSum = 0;
    for (int i = 0; i < k; i++) {// Step 1: sum of first window
        windowSum += a[i];
    }
    int best = windowSum;
    // Store indices of best window
    int bestLeft = 0;
    int bestRight = k - 1;
    // Step 2: slide the window
    for (int right = k; right < n; right++) {
        int left = right - k;
        windowSum += a[right] - a[left];
        if (windowSum > best) {
            best = windowSum;
            bestLeft = left + 1;
            bestRight = right;
        }
    }
    cout << "Indices = [" << bestLeft << ", " << bestRight << "]" << endl;
    return best; }
```

Key Points

- Compute first window's sum once — $O(K)$.
- Each new window reuses the previous sum.
- $\text{left} = \text{right} - k$ gives the element leaving.
- Total time: $O(K) + O(n - K) = O(n)$.
- Space: $O(1)$ — no extra array needed.

Variable-Size Sliding Window — Theory

Used when the window size isn't fixed — instead we grow/shrink it based on a condition (e.g. "sum $\leq$ target", "no repeated characters").

1

1. Expand

Move right pointer forward, add $A[\text{right}]$ into the window.

2

2. Shrink

While the condition is violated, move left pointer forward, remove $A[\text{left}]$.

3

3. Update

After fixing the window, record the answer (length, sum, count...).

Complexity: $O(n)$ — left pointer moves forward at most n times total, never backward.

Two Pointers: left & right

1	2	1	0	1	1	2
---	---	---	---	---	---	---

▲
left

▲
right

Window $[\text{left}, \text{right}]$ satisfies the condition — grow right to explore further, shrink left only when it breaks.

Common uses: longest substring without repeats, smallest subarray with sum $\geq S$, longest subarray with at most K distinct values.

Worked Example — Longest Subarray, Sum ≤ 8

Array: [2, 1, 3, 4, 1, 2] Find the longest subarray whose sum ≤ 8 .

2	1	3	4	1	2
---	---	---	---	---	---

right=2: window[2,1,3] sum=6 ≤ 8 OK. len=3

2	1	3	4	1	2
---	---	---	---	---	---

right=3: sum=10 $> 8 \rightarrow$ shrink left

2	1	3	4	1	2
---	---	---	---	---	---

after shrink: window[1,3,4] sum=8 ≤ 8 OK. len=3

2	1	3	4	1	2
---	---	---	---	---	---

right=5: sum=10 $> 8 \rightarrow$ shrink left

2	1	3	4	1	2
---	---	---	---	---	---

window[4,1,2] sum=7 ≤ 8 OK. len=3

Answer: longest length = 3 (multiple windows achieve it)

Variable-Size Sliding Window — C++ Code

```
int longestSubarray(vector<int>& a, int target) {  
    int n = a.size();  
    int left = 0, sum = 0, best = 0;  
  
    for (int right = 0; right < n; right++) {  
        sum += a[right];           // expand  
  
        while (sum > target) {      // shrink  
            sum -= a[left];  
            left++;  
        }  
  
        best = max(best, right - left + 1);  
    }  
    return best;  
}
```

Key Points

- right always moves forward — one pass.
- while (not the safer if) keeps shrinking until valid.
- left never resets — total left moves $\leq n$.
- window length = right - left + 1.
- Amortized $O(n)$: each index visited $O(1)$ times by each pointer.

Kadane's Algorithm — Intuition

Goal: find the contiguous subarray with the maximum sum — array can contain negative numbers, so sliding window's simple shrink rule doesn't directly apply.

Key Insight

At every index, ask: *"Should I extend the previous subarray, or start fresh here?"*

```
currentSum = max( A[i], currentSum + A[i] )
```

If currentSum drops below A[i] alone, **abandon it** and start a new subarray at i — a negative prefix only drags the sum down.

Complexity: **O(n) time, O(1) space** — a single left-to-right scan.

At Each Step

`currentSum + A[i]`

vs

`A[i] (start fresh)`

Take the larger → that's the new currentSum.

Track bestSum = max(bestSum, currentSum) across the whole scan.

Worked Example — With Negative Numbers

Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

i	0	1	2	3	4	5	6	7	8
A[i]	-2	1	-3	4	-1	2	1	-5	4
currentSum	-2	1	-2	4	3	5	6	1	5
bestSum	-2	1	1	4	4	5	6	6	6

At $i=2$ (-3): $\text{currentSum} = \max(-3, 1 + -3 = -2) = -2$ (still negative, keep old chain since $-2 > -3$)

At $i=4$ (-1): $\text{currentSum} = \max(-1, 4 + -1 = 3) = 3 \rightarrow$ extending [4, -1] beats restarting

At $i=7$ (-5): currentSum drops to 1, but bestSum already locked in 6 from $i=6$ — best is never lost.

Maximum subarray sum = 6 (subarray [4, -1, 2, 1], indices 3..6)

Kadane's Algorithm — C++ Code

```
int maxSubArray(vector<int>& a) {  
    int n = a.size();  
    int currentSum = a[0];  
    int bestSum = a[0];  
  
    for (int i = 1; i < n; i++) {  
        // extend or restart here  
        currentSum = max(a[i], currentSum +  
a[i]);  
  
        // remember the best seen so far  
        bestSum = max(bestSum, currentSum);  
    }  
    return bestSum;  
}
```

Key Points

- Initialize both sums with $a[0]$, not 0 — handles all-negative arrays correctly.
- One pass, $O(n)$ time, $O(1)$ space.
- To also report the subarray indices, track a start index when restarting.
- If the array can be empty, define the answer for that case separately.

Which Technique Do I Use?

Fixed Sliding Window	Variable Sliding Window	Kadane's Algorithm
USE WHEN Window size K is given	USE WHEN Find longest/shortest window meeting a condition	USE WHEN Maximize sum over ALL possible subarray lengths
EXAMPLES <i>Max/min sum of size K, moving average</i>	EXAMPLES <i>Longest substring w/o repeats, smallest sum $\geq S$</i>	EXAMPLES <i>Max subarray sum, max profit (single transaction)</i>
Time: $O(n)$ Space: $O(1)$	Time: $O(n)$ Space: $O(1)$	Time: $O(n)$ Space: $O(1)$

Rule of thumb: fixed-size $K \rightarrow$ Fixed Window. Contiguous elements can be negative and no size constraint $\rightarrow$ Kadane's. Otherwise, growing/shrinking a window $\rightarrow$ Variable Window.

Maximum Subarray

Problem Statement

Given an integer array `nums`, find the contiguous subarray (containing at least one number) which has the largest sum, and return that sum.

Technique

Kadane's Algorithm

Approach

1. Scan the array once, maintaining `currentSum` and `bestSum`.
2. At each index: `currentSum = max(nums[i], currentSum + nums[i])`.
3. Update `bestSum = max(bestSum, currentSum)`.
4. Return `bestSum` after the loop.

Example

Input:

`[-2, 1, -3, 4, -1, 2, 1, -5, 4]`

Output:

6

Explanation:

`[4, -1, 2, 1]` has the largest sum = 6.

Judge tags: array, dp, divide & conquer

Constraints: $1 \leq n \leq 10^5$

Maximum Sum of Size K

Problem Statement

Given an array of positive integers and an integer K, find the maximum sum of any contiguous subarray of size exactly K.

Technique

Fixed Sliding Window

Approach

1. Compute the sum of the first K elements as the initial window.
2. Slide right: add the new element, subtract the one leaving.
3. Track the maximum sum seen across all windows.
4. Return the maximum after scanning the array once.

Example

Input:

`[2, 1, 5, 1, 3, 2]`, $K = 3$

Output:

9

Explanation:

Window [5,1,3] gives the max sum = 9.

Judge tags: array, sliding window

Constraints: $1 \leq K \leq n \leq 10^5$

Longest Subarray Problems

Problem Statement

Given an array of positive integers and a target sum S , find the length of the longest contiguous subarray whose sum is $\leq S$.

Technique

Variable Sliding Window

Approach

1. Expand the window by moving right and adding $a[\text{right}]$.
2. While $\text{sum} > S$, shrink from the left until valid again.
3. After each expand+shrink, update $\text{best} = \max(\text{best}, \text{right} - \text{left} + 1)$.
4. Since elements are positive, both pointers move forward only — $O(n)$.

Example

Input:

$[2, 1, 3, 4, 1, 2], \quad S = 8$

Output:

3

Explanation:

$[2, 1, 3]$ or $[4, 1, 2]$ both have length 3 with sum ≤ 8 .

Related pattern: also solves "longest substring with at most K distinct characters" using a frequency map instead of a sum.

Homework — Solve 5 to 8 Problems

Practice makes the pattern automatic. Solve at least 5 of the following (aim for all 8) before next lab session.

1

Maximum Average Subarray (Leetcode)

Fixed Window

2

Minimum Size Subarray Sum (Leetcode)

Variable Window

3

Longest Substring Without Repeating Characters (Leetcode)

Variable Window

4

Best Time to Buy and Sell Stock (Leetcode)

Kadane's (variant)

5

Maximum Sum Circular Subarray (Leetcode)

Kadane's (extended)

6

Subarrays with K Different Integers (Leetcode)

Variable Window

7

Grumpy Bookstore Owner (Leetcode)

Fixed Window

8

Maximum Number of Vowels in a Substring of Given Length (Leetcode)

Fixed Window

Key Takeaways



Brute force is $O(n^2)$ or worse

Checking every subarray directly doesn't scale for n up to 10^5 – 10^6 .



Fixed Window → constant-size problems

Slide by removing the left element and adding the right one: $O(n)$.



Variable Window → condition-based length

Expand right, shrink left only when needed — left never moves back.



Kadane's → best sum over ANY length

At each step, extend or restart: $\text{currentSum} = \max(A[i], \text{currentSum} + A[i])$.

Problems to practice from Codeforces:

1840C — Ski Resort (Variable Size Sliding Window)

580A — Kefa and First Steps

1690D — Black and White Stripe

Next step: solve today's homework, then bring questions to the next lab session.